

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Observation Task No.	3
Student Name	B.JATIN
Roll No.	A24126552070
Date	21-09-2026

1. TITLE

Express.js Student Server

2. AIM

To create a basic Express.js server with the routes `/`, `/students` and `/about` and demonstrate HTTP methods, routing and response handling.

3. OBJECTIVE

1. To set up an Express.js application.
2. To create and start an HTTP server.
3. To implement GET routes.
4. To return a list of at least five students as JSON.
5. To display application information through an `/about` route.
6. To understand routing and response handling in Express.js.
7. To verify the server through the browser.

4. THEORY

Express.js is a Node.js web framework used to create servers and APIs. Routes map HTTP methods and URL paths to handler functions. The GET method is suitable for retrieving information. In this application, the home route returns HTML, `/students` returns JSON data for five students, and `/about` returns information about the application.

Application Flow: Start Express Server → Receive GET Request → Match Route → Execute Handler → Send Response

5. ALGORITHM / PROCEDURE

1. Create a Node.js project and install Express.js.
2. Import Express and create an application object.
3. Set the server port to 3000.
4. Create the `/` route using `app.get()`.
5. Create an array containing five student records.
6. Create the `/students` GET route and return JSON.
7. Create the `/about` GET route and return application information.
8. Start the server with `app.listen()`.
9. Open the routes in a browser and verify the responses.

6. PROGRAM

```
1: const express = require("express");
2:
3: const app = express();
4: const PORT = 3000;
5:
6: // Middleware to parse JSON request bodies.
7: app.use(express.json());
8:
9: // Home route.
10: app.get("/", (req, res) => {
11:   res.send(`
12: <h1>Student Management Server</h1>
13: <p>Welcome to the Express.js Student Server.</p>
14: <p>Available routes:</p>
15: <ul>
16: <li></li>
17: <li>/students</li>
18: <li>/about</li>
19: </ul>
20: `);
21: });
22:
23: // Student data.
24: const students = [
25:   { id: 1, name: "Jatin", course: "CSE", year: 3 },
26:   { id: 2, name: "Rahul", course: "CSE", year: 3 },
27:   { id: 3, name: "Priya", course: "ECE", year: 3 },
28:   { id: 4, name: "Arjun", course: "IT", year: 3 },
29:   { id: 5, name: "Sneha", course: "CSE", year: 3 }
30: ];
31:
32: // Return the student list as JSON.
33: app.get("/students", (req, res) => {
34:   res.json(students);
35: });
36:
37: // About route.
38: app.get("/about", (req, res) => {
39:   res.send(`
40: <h1>About the Application</h1>
41: <p>This is a basic Express.js Student Server.</p>
42: <p>It demonstrates routing, HTTP methods and response handling.</p>
43: `);
44: });
45:
46: // Start the server.
47: app.listen(PORT, () => {
48:   console.log(`Server running at http://localhost:${PORT}`);
49: });
```

7. CODE EXPLANATION

Code / Syntax	Explanation
require('express')	Imports the Express.js framework.
express()	Creates an Express application instance.
app.get()	Defines a route that handles HTTP GET requests.
res.send()	Sends an HTML or text response to the client.
res.json()	Sends JavaScript data as a JSON response.
app.listen()	Starts the Express server on the specified port.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Open /	Home page displayed	Student Management Server displayed	Pass
TC-02	Open /students	Five students returned	Student JSON returned	Pass
TC-03	Open /about	Application information displayed	About response displayed	Pass
TC-04	Start server on port 3000	Server starts	Server running at localhost:3000	Pass

9. OUTPUT

Browser output for the Express.js home route at localhost:3000, showing the available routes.



Student Management Server

Welcome to the Express.js Student Server.

Available routes:

- /
- /students
- /about

10. RESULT

Thus, the Express.js Student Server was successfully developed. The application demonstrates Express setup, GET routing, JSON response handling, HTML response handling and server execution on port 3000.